

RBAC Review

Detailed review of Kubernetes ServiceAccount permissions, allowed and denied actions, privilege escalation simulation, remediation, and CIS Benchmark alignment.

Field	Value
Document Type	RBAC Assessment
Author	Jagriti Banerjee
Project Scope	Private Kubernetes DevSecOps lab using Docker, Kind, Calico, Trivy, Falco, OPA/Gatekeeper, ArgoCD, and kube-bench
Audience	Security engineers, DevSecOps reviewers, recruiters, and technical interviewers
Classification	Private lab / portfolio evidence

Reviewer snapshot

Review Dimension	Included in Document
Technical depth	Control rationale, validation commands, expected results, risk interpretation, and remediation guidance.
Evidence model	Screenshot references, manifest fields, scanner outputs, and repeatable kubectl commands.
Professional use	Can be used as a GitHub portfolio artifact, interview talking point, and internal-style security record.
CIS alignment	Includes a dedicated CIS Kubernetes Benchmark section with lab-context interpretation.

Scope statement

This document is based on controlled, authorized testing performed inside a private lab environment. It does not represent a production certification and does not include public or third-party targets. The focus is professional evidence, repeatable validation, and security engineering reasoning.

Spacing and readability note

The document is intentionally laid out with compact paragraphs, split tables, and dedicated evidence pages so that detail is preserved without crowded pages or large blank gaps.

Document coverage

Coverage Area	Included
Assessment narrative	Executive summary, scope, methodology, technical analysis, and validation interpretation.
Evidence references	Relevant screenshots, command outputs, reports, and expected retest outcomes.
Remediation planning	Priority-based actions, production notes, and follow-up recommendations.
Benchmark context	CIS Kubernetes Benchmark alignment and lab-specific interpretation.

1. Executive Summary

The RBAC review confirms that the production-style demo-app ServiceAccount follows least-privilege principles while also documenting an intentional overprivileged ServiceAccount scenario used for education. The document explains how cluster-admin binding creates cluster compromise risk and how namespace-scoped RoleBinding remediation limits blast radius.

2. RBAC Objects Reviewed

Object	Name	Scope	Purpose
ServiceAccount	demo-app-sa	devsecops	Workload identity used by the hardened application.
Role	demo-app-role	devsecops	Allows get/list on ConfigMaps only.
RoleBinding	demo-app-rolebinding	devsecops	Connects demo-app-sa to demo-app-role.
ServiceAccount	overprivileged-sa	rbac-lab	Intentional attack simulation identity.
ClusterRoleBinding	pwned-overprivileged-sa	cluster-wide	Intentional unsafe binding to cluster-admin for demonstration.

3. Least-Privilege Validation

```
kubectl auth can-i list configmaps --as=system:serviceaccount:devsecops:demo-app-sa -n devsecops
kubectl auth can-i get secrets --as=system:serviceaccount:devsecops:demo-app-sa -n devsecops
kubectl auth can-i list nodes --as=system:serviceaccount:devsecops:demo-app-sa
```

Action	Expected	Observed Security Meaning
list configmaps in namespace	yes	Application has only the minimum read access used for configuration.
get secrets in namespace	no	Sensitive values are protected from the workload identity.
list nodes cluster-wide	no	Workload identity has no cluster reconnaissance capability.
create clusterrolebindings	no	Workload cannot grant itself or others cluster-admin access.

4. Privilege Escalation Simulation

An intentionally unsafe ClusterRoleBinding was created to demonstrate how excessive RBAC can become a full cluster compromise path. This was performed only inside the private lab.

```
kubectl create clusterrolebinding pwned-overprivileged-sa --clusterrole=cluster-admin
--serviceaccount=rbac-lab:overprivileged-sa
kubectl auth can-i '*' '*' --as=system:serviceaccount:rbac-lab:overprivileged-sa --all-namespaces
kubectl auth can-i get secrets --as=system:serviceaccount:rbac-lab:overprivileged-sa --all-namespaces
```

4.1 Attack impact

- A compromised ServiceAccount token could be used to read Kubernetes secrets.
- The attacker could create new ClusterRoleBindings and persist administrative access.
- The attacker could deploy privileged pods and attempt host access.
- The attacker could modify or delete workloads and security controls.
- ClusterRoleBinding misuse increases blast radius from one namespace to the full cluster.

5. Remediation

```
kubectl delete clusterrolebinding pwned-overprivileged-sa
kubectl create role read-configmaps-only --verb=get,list --resource=configmaps -n rbac-lab
kubectl create rolebinding overprivileged-sa-read-configmaps-only --role=read-configmaps-only
--serviceaccount=rbac-lab:overprivileged-sa -n rbac-lab
```

Before	After
ClusterRoleBinding to cluster-admin grants full cluster power.	Namespace RoleBinding grants only get/list ConfigMaps in rbac-lab.
Can get secrets in all namespaces.	Cannot get secrets in namespace or cluster-wide.
Can create additional ClusterRoleBindings.	Cannot create ClusterRoleBindings.

6. CIS Kubernetes Benchmark Section

CIS describes the Kubernetes Benchmark as consensus-based secure configuration guidance for Kubernetes. In this lab, CIS Benchmark alignment is used as an assessment reference and learning framework. Because the cluster is a Kind-based private lab rather than a production kubeadm or managed service environment, benchmark results are interpreted with environment context. The expected professional practice is to document PASS, FAIL, WARN, and not-applicable outcomes; assign owners to applicable failures; retest after remediation; and keep benchmark artifacts in the evidence folder.

For RBAC, the benchmark-aligned review principle is to avoid unnecessary cluster-admin permissions, restrict service account privileges, and document why each permission exists. This lab validates those principles through explicit allow/deny checks.

7. Evidence Highlights

8. Retest Checklist

Test	Pass Criteria
Application can list ConfigMaps	kubectl auth can-i returns yes only for expected namespace.
Application cannot get Secrets	kubectl auth can-i returns no.
Application cannot list Nodes	kubectl auth can-i returns no.
No unexpected cluster-admin bindings	ClusterRoleBinding audit shows no application ServiceAccount bound to cluster-admin.
No token automount	Deployment and ServiceAccount do not automatically mount API token.

Quality Assurance and Evidence Preservation

QA Item	Required Practice
Evidence naming	Use stable screenshot names and link each item to a control or retest step.
Command repeatability	Store commands in documentation so another reviewer can reproduce validation.
Exception handling	Any accepted risk should include owner, reason, expiry date, and compensating control.
Retest discipline	Do not close a remediation item until the expected negative and positive tests both pass.
CI/CD linkage	Where possible, convert manual validation into security gates or scheduled checks.

Reviewer notes for this Kubernetes security area

This document intentionally combines implementation evidence with risk interpretation. The objective is to show not only that commands were executed, but also why each control matters, how it reduces attack surface, and how the result should be retested after future changes.

References and Standards Basis

Reference	Use in this Document
CIS Kubernetes Benchmark	CIS describes its Kubernetes Benchmark as a product of a community consensus process with secure configuration guidelines for Kubernetes.
Kubernetes RBAC documentation	Kubernetes RBAC uses Role, ClusterRole, RoleBinding, and ClusterRoleBinding objects to manage authorization.
Kubernetes NetworkPolicy documentation	NetworkPolicy governs how pods communicate with each other and with network endpoints.
Kubernetes Pod Security Standards	The restricted profile is intended for strongly hardened workload execution.

Reference URLs used during document preparation: <https://www.cisecurity.org/benchmark/kubernetes>, <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>, <https://kubernetes.io/docs/concepts/services-networking/network-policies/>, and <https://kubernetes.io/docs/concepts/security/pod-security-standards/>.